Patricia Fernández Sanz

Automated Power and Rate Control in 802.11 Wireless Networks

Introduction

The deployments of a wireless network today have two important properties, they are unplanned, with most access points deployed by users spontaneously and they are unmanaged, since manually configuring and managing a wireless network is very complicated.

Most APs are not configured to minimize interference with their neighbors, they perform extensive simulations to show that the performance of end clients could suffer significantly in such wireless deployments.

To improve said experience, PARF was introduced as an algorithm to minimize interference between neighboring APs, while ensuring robust end client performance.

In this report we will walk through how to implement a modified version of PARF in which the transmit power level is incremented by a PowerStepSize as an attribute instead of 1.

With this change we expect higher throughput when end clients move frequently but the average transmit power will also be higher.

We will first evaluate its performance through different simulations, extract metrics and visualize and analyze its performance and results.

Modifications in the simulation

We were given a sample code to modify in order to make it work for our simulations. These are the modifications that we did to it:

1.

Depending on the simulation, we would change the values on the main function in the wifi_template.cc code. For the cases in Simulation 1, the speed of the MS will always be 1 but we would change the step size, changing between 1, 3 and 5.

2.

We also did some modifications in parf-wifi-managers. We added a new variable, powerStepSize that we manually change from 1 to 3 to 5 instead of adding 1 as it did before. This was required for the simulations.

Running the simulations

To run the simulations we used `./ns3 run scratch/wifi_template.cc 2> tx_power_x_y.txt` so the .pcap would be generated and we had the .txt files that we would use later on to calculate the transmission power.

Depending on the simulation, we would change the x value for the speed value and the y value for the step size value.

In order to later on do the other graphs and analyze the collected data, we converted the results to a .csv output file with the command `tshark -r wifi-n0-i1.pcap -Tfields -e frame.time_relative -e udp.srcport -e udp.dstport -e frame.len -E separator=, > wifi_sim_ms_x_y.csv` where we would change again x and y depending on the simulation that we were running and also the .pcap, since for the metrics we need MS .pcap (n0_i1) and AP .pcap (n1_i1)

Process the traces

To process the traces we wrote a python code. On the simulation code we generated from the different PCAP files two .csv, one for AP and another one for MS. We also downloaded the .txt file generated which contains other measurements that we will use in order to calculate the transmission power.
Once we had these three files for each simulation, we created a new code in python to process these metrics and graph them.

For the throughput calculations, we implemented a sliding window that iterates through time. For each window it sums the TCP segment lengths of all packets within that window and finally converts the total bytes to Mbps. We also calculated the average throughput of each step size giving a fixed velocity to compare them later on.

For the loss rate calculations, we also implemented a sliding window so it's easier and the differences are more noticeable. For it, we calculated it roughly by dividing the result of bytes sent minus bytes received by bytes sent. For it, we need both .pcap. Since it was an approximation, we did not match individually each packet sent with each packet received.

Finally, for the transmit power, since it was printed to the console whenever it was adjusted, we downloaded the .txt file. We splitted it into parts and saved the time and power parts and appended them and created a dataframe to work with. In order to facilitate the comprehension of the plot, we arranged them and implemented a sliding window, as well as the average transmit power for each step size.

Once all those calculations were done, we plotted all of them by creating one subplot for each figure and added labels and grids for clarity.

Running the script to process the trace

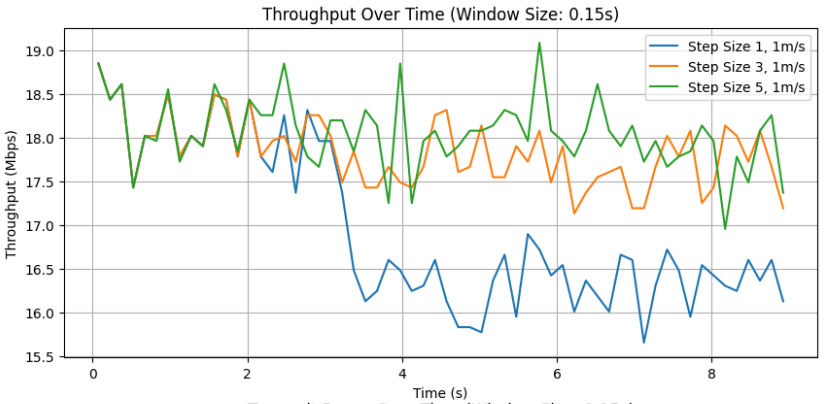
To process the traces, when we first run the code, it will create the .txt file directly since we put that in the run command. Once we run the tshark command we will have the .csv file with metrics.
Those files are then used on the python program explained above. In order to run them, we just have to change the name of the file name and its location on our local machine to run them.

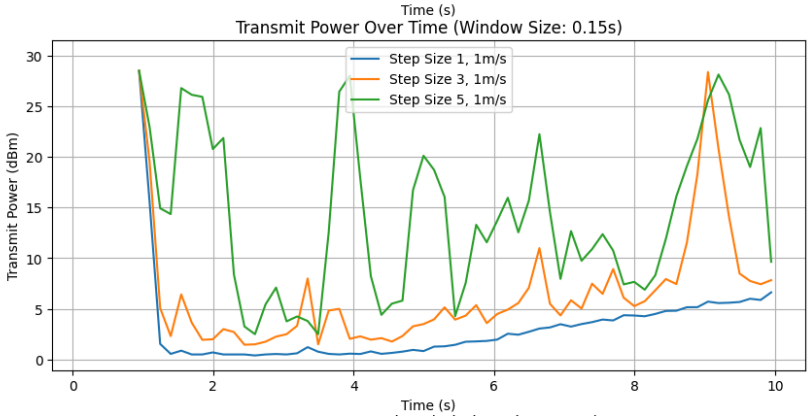
Simulation Velocity 1

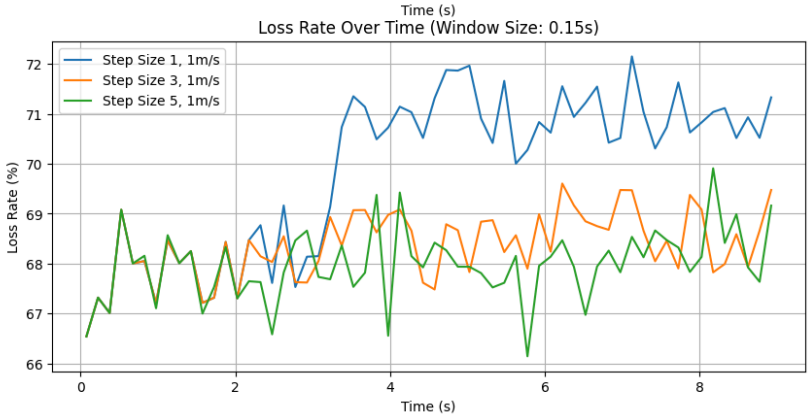
Simulation 1 Step Size 1
Average Throughput: 16.97 Mbps
Average Transmit Power: 3.32 dBm
Average Packet Loss Rate: 69.88%

Simulation 1 Step Size 3
Average Throughput: 17.84 Mbps
Average Transmit Power: 6.66 dBm
Average Packet Loss Rate: 68.36%

Simulation 1 Step Size 5
Average Throughput: 18.05 Mbps
Average Transmit Power: 14.57 dBm
Average Packet Loss Rate: 67.98%







Given the plots achieved with a velocity of v = (1, 0, 0) and the different step sizes, we are able to infer different things.

Focusing on the throughput, as we predicted, its average increases with larger step sizes. Step size 5 as the highest average throughput. It can also be seen that the step size 5 has more peaks than the other two, this means that it can achieve a higher throughput at certain times.

The transmit power also increases with larger step sizes. Looking at the over time graph, it can be seen again that step size 5 has a lot more peaks while step size 1 barely does not have any peak.

Finally, for the loss rate, the average packet loss rate decreases with larger step sizes. While step size 1 has some pretty noticeable peaks, step size 5 has lower peaks.

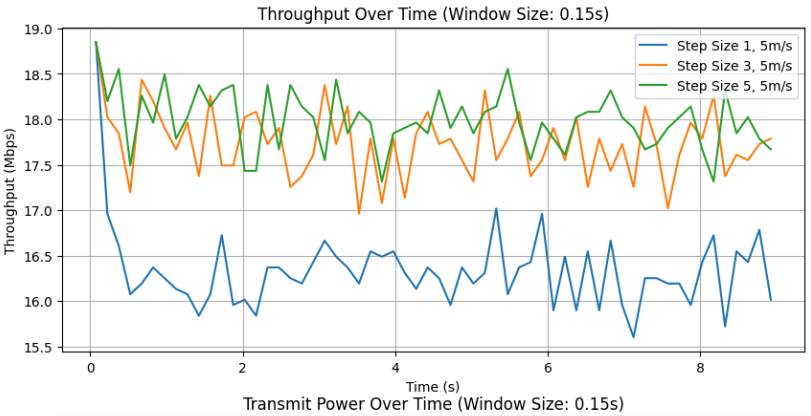
The results demonstrate a clear tradeoff between throughput and transmit power. Increasing the step size leads to higher throughput and lower packet loss but at the cost of significantly higher transmit power. At a velocity of 1m/s, the client is moving relatively slowly, however, even at this speed, the PARF algorithm is able to improve performance by dynamically adjusting the transmit power.
Step size 5 shows the best results in terms of throughput and loss rate, but it also has the biggest transmit power, meaning that it will drain the battery of the MS faster than the other step sizes.

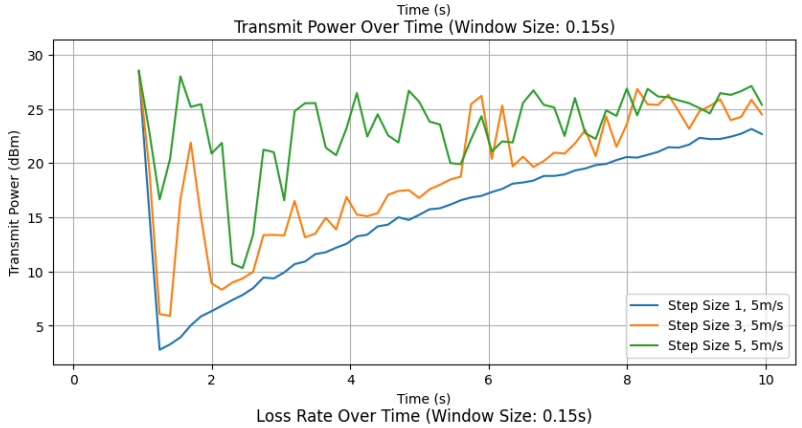
Simulation Velocity 5

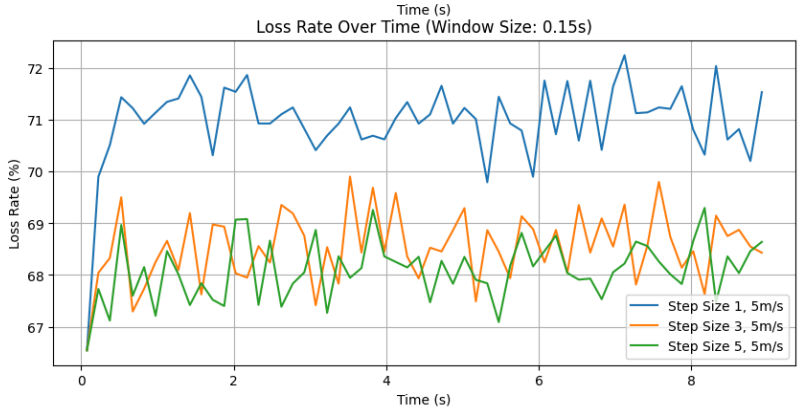
Simulation 5 Step Size 1
Average Throughput: 16.33 Mbps
Average Transmit Power: 15.37 dBm
Average Packet Loss Rate: 70.99%

Simulation 5 Step Size 3
Average Throughput: 17.74 Mbps
Average Transmit Power: 18.82 dBm
Average Packet Loss Rate: 68.54%

Simulation 5 Step Size 5
Average Throughput: 17.99 Mbps
Average Transmit Power: 23.17 dBm
Average Packet Loss Rate: 68.09%







In this case, our plots are achieved with a velocity v = (5, 0, 0).

The throughput is similar to the 1m/s scenario, it increases with larger step sizes. Step size 5 has the highest average throughput once again but this time we can see that for step size 1 is very low at the beginning of the simulation but then it increases and maintains a more stable output.

The average transmit power also increases with larger step sizes. In this case, the transmit power of step size 1 increases over time, which differs from the previous simulation, although it is more stable that the other two step sizes.

Once again, the average packet loss rate decreases with larger step sizes, having again step size 5 and 3 almost a 2% difference with step size 1.

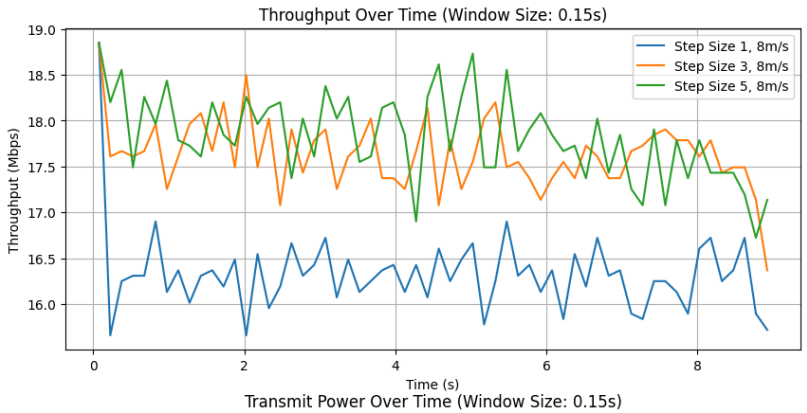
Comparing these results with those achieved in the previous simulation, we see that the average transmit power is significantly higher across all step sizes at 5m/s. This indicates that the PARF algorithm is working harder to maintain a stable connection at the higher velocity.
The tradeoff between throughput and transmit power is even more pronounced in this simulation, the higher velocity demands more transmit power to maintain a reasonable level of performance.

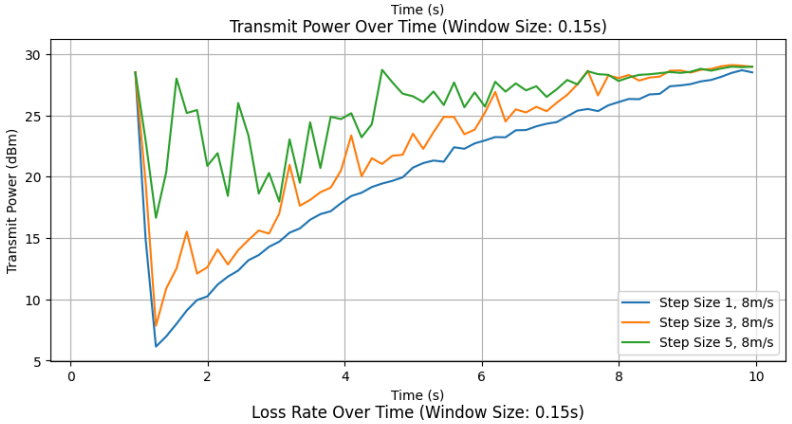
Simulation Velocity 8

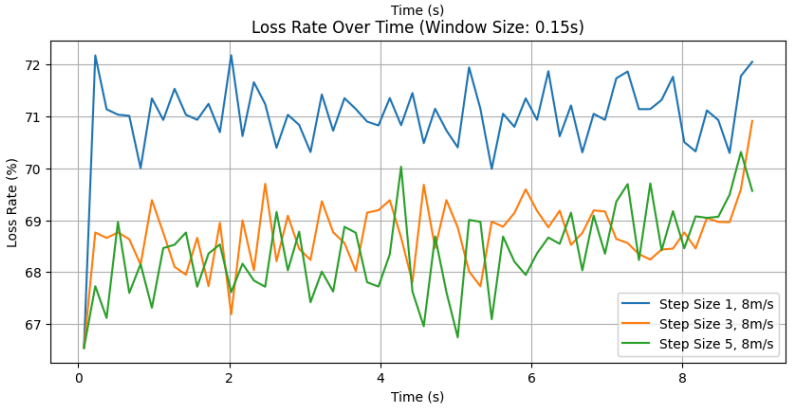
Simulation 8 Step Size 1
Average Throughput: 16.33 Mbps
Average Transmit Power: 20.46 dBm
Average Packet Loss Rate: 71.00%

Simulation 8 Step Size 3
Average Throughput: 17.63 Mbps
Average Transmit Power: 22.48 dBm
Average Packet Loss Rate: 68.71%

Simulation 8 Step Size 5
Average Throughput: 17.82 Mbps
Average Transmit Power: 25.66 dBm
Average Packet Loss Rate: 68.38%







Finally, we will discuss the results obtained from the simulation with a velocity of v= (8, 0, 0), the fastest out of the three simulations.

Once again, the trend continues, throughput increases with larger step sizes, the average transmit power also increases with larger step sizes and the average packet loss rate decreases.

At 8m/s, the average transmit power is even higher than it was at 5m/s, indicating the PARF algorithm is basically working even harder to maintain connectivity. The throughput of the step size 1 is the same that it was on the previous simulation but the transmit power is higher.

The transmit power now shows that the step size 1 algorithm is struggling to maintain a strong signal at the beginning of the simulation, needing to increase the power over time. The high speed of the MS is generating a lot of changes in the channel conditions.

The tradeoff between the throughput and the transmit power in this simulation is the most significant, since the energy cost of maintaining a high throughput is very high.

Conclusion

Higher velocities demand significantly more transmit power to maintain a stable connection. Increasing the step size continues to improve performance, but the energy consumption implications are increasingly severe.

The step size 1 shows that it has a lot of problems to maintain stable connections at the beginning of the simulations and at high velocities, like on the third simulation, the energy consumption of the MS is a very important factor to take into account, since it drastically increases and affects.

It's clear that while larger step sizes provide better performance in terms of throughput and packet loss, they come with a significant energy cost. This highlights the need for more energy efficient algorithms, especially in cases or scenarios where there is high client mobility.